AWS RDS Deployment (Floodify Project)

What is RDS Deployment?

RDS deployment means deploying the application database on Amazon RDS (Relational Database Service) instead of running the database directly on the EC2 server.

Step 1: Create a VPC for RDS and EC2 Deployment

Before launching the EC2 instance and configuring the RDS database, create a **Virtual Private Cloud (VPC)**. The VPC provides the network environment in which the EC2 instance and RDS database can communicate securely.

1. Open the VPC Console -

1. Log in to the **AWS Management Console**.
2. Search for **VPC** in the AWS search bar.
3. Select **VPC**.
4. From the left navigation menu, click **Your VPCs**.
5. Click **Create VPC**.

2. Select Resources to Create

Under **Resources to create**, select: **VPC and more**

This option automatically creates the required networking components such as:

- VPC
- Public subnets
- Private subnets
- Route tables
- Internet Gateway
- Network configuration

3. Configure the VPC Settings - Use the following configuration:

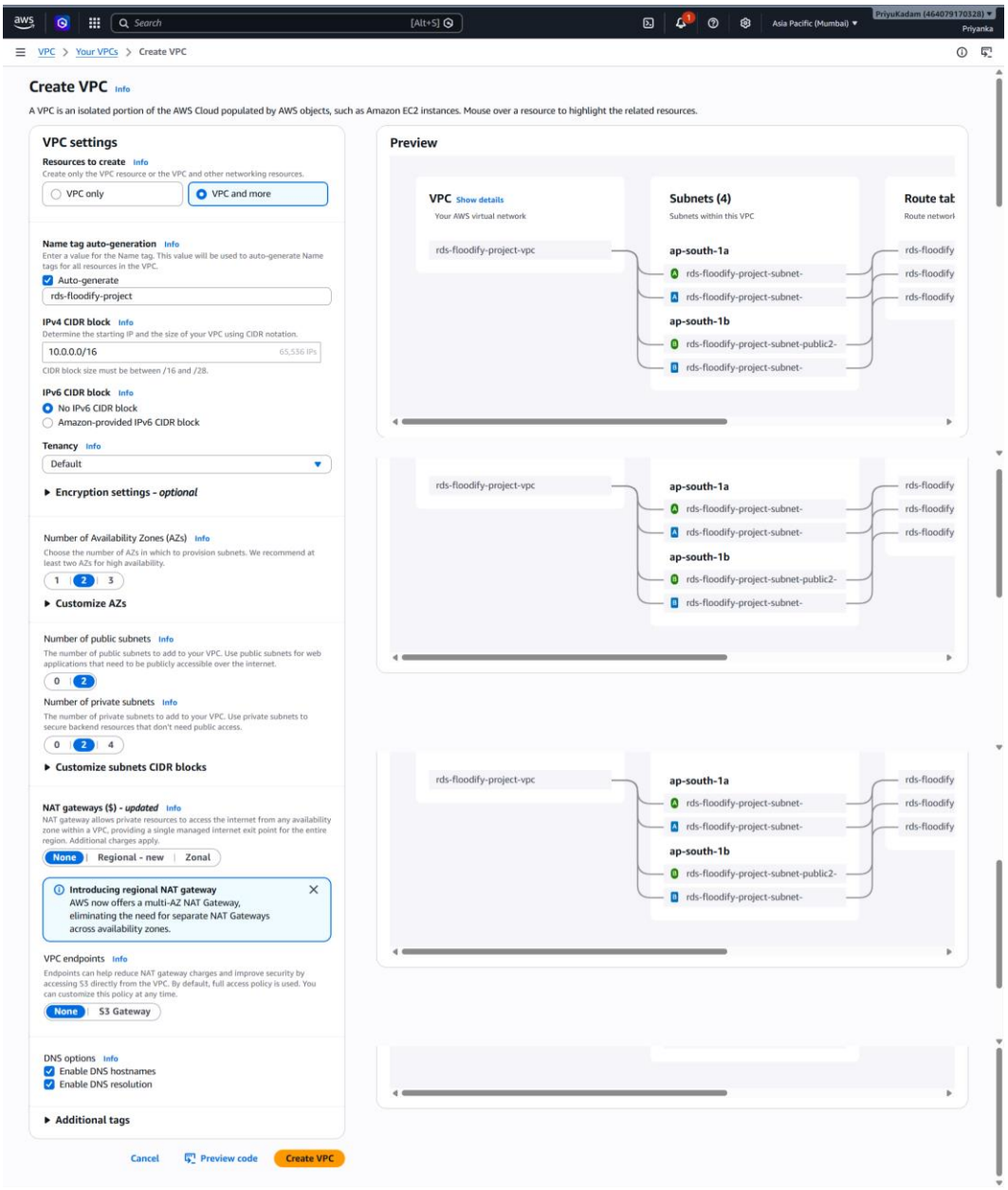
Setting	Value
Name tag auto-generation	`rds-floodify-project`
IPv4 CIDR block	`10.0.0.0/16`
IPv6 CIDR block	No IPv6 CIDR block
Tenancy	Default
Number of Availability Zones (AZs)	2
Number of public subnets	2
Number of private subnets	2
NAT Gateways	None
VPC endpoints	None
Enable DNS hostnames	Enabled
Enable DNS resolution	Enabled

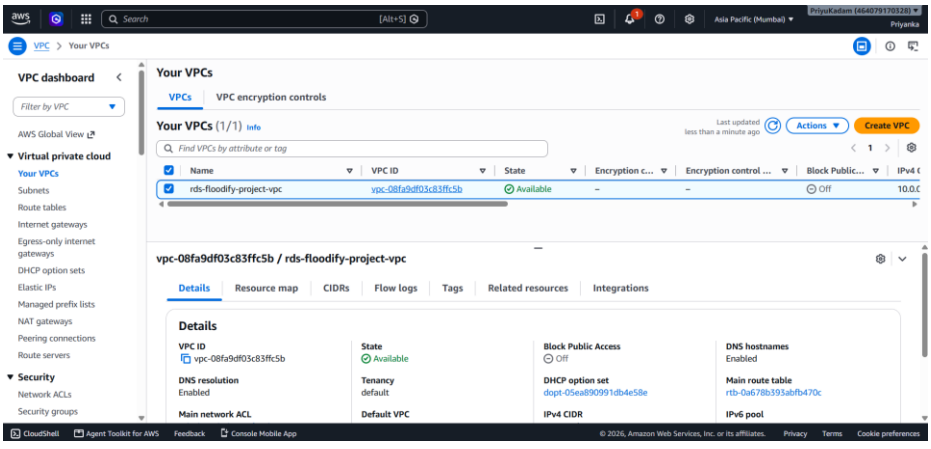
4. Create the VPC

After verifying the configuration:

1. Click **Create VPC**.
2. Wait until AWS completes the creation process.

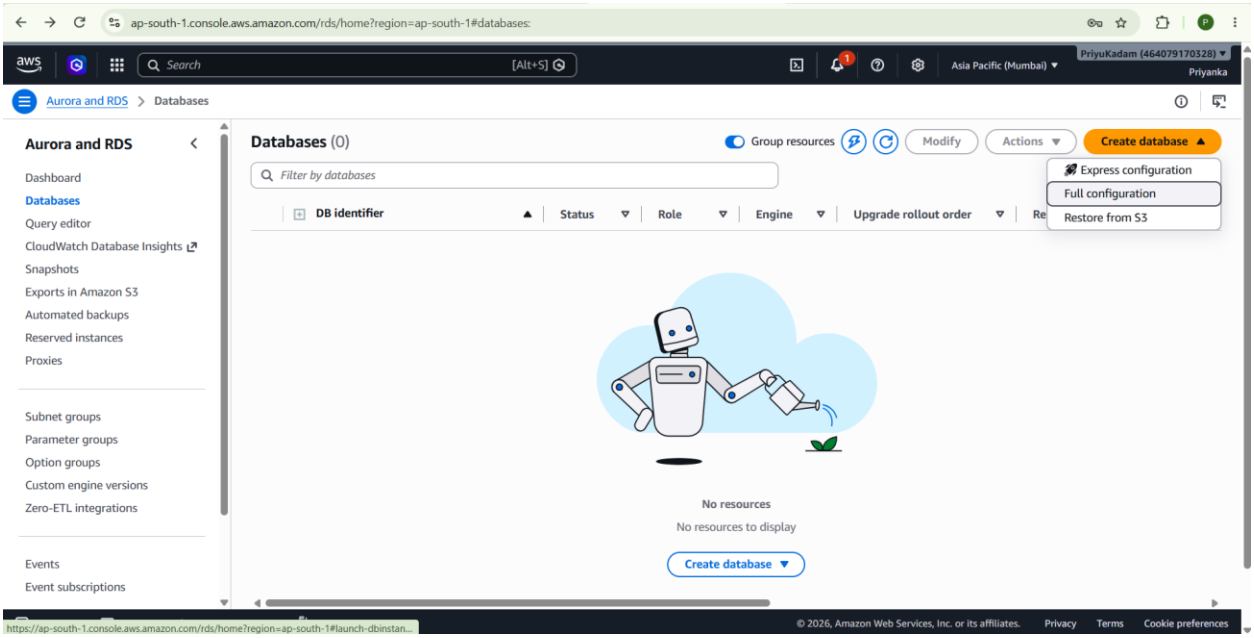
The VPC will contain public and private subnets across two Availability Zones.


Search
[Alt+S]
Asia Pacific (Mumbai)
PriyuKadam (464079170328)
Priyanka
VPC > Your VPCs > Create VPC
Create VPC Info
A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances. Mouse over a resource to highlight the related resources.
VPC settings
Resources to create Info
Create only the VPC resource or the VPC and other networking resources.
VPC only
VPC and more
Name tag auto-generation Info
Enter a value for the Name tag. This value will be used to auto-generate Name tags for all resources in the VPC.
Auto-generate
rds-floodify-project
IPv4 CIDR block Info
Determine the starting IP and the size of your VPC using CIDR notation.
10.0.0.0/16
65,536 IPs
CIDR block size must be between /16 and /28.
IPv6 CIDR block Info
No IPv6 CIDR block
Amazon-provided IPv6 CIDR block
Tenancy Info
Default
Encryption settings - optional
Number of Availability Zones (AZs) Info
Choose the number of AZs in which to provision subnets. We recommend at least two AZs for high availability.
1 2 3
Customize AZs
Number of public subnets Info
The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.
0 2
Number of private subnets Info
The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.
0 2 4
Customize subnets CIDR blocks
NAT gateways ($) - updated Info
NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.
None Regional - new Zonal
Introducing regional NAT gateway
AWS now offers a multi-AZ NAT Gateway, eliminating the need for separate NAT Gateways across availability zones.
VPC endpoints Info
Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.
None S3 Gateway
DNS options Info
Enable DNS hostnames
Enable DNS resolution
Additional tags
Cancel
Preview code
Create VPC
Preview
VPC Show details
Your AWS virtual network
rds-floodify-project-vpc
Subnets (4)
Subnets within this VPC
ap-south-1a
rds-floodify-project-subnet-
rds-floodify-project-subnet-
ap-south-1b
rds-floodify-project-subnet-public2-
rds-floodify-project-subnet-
Route tab
rds-floodify
rds-floodify
rds-floodify

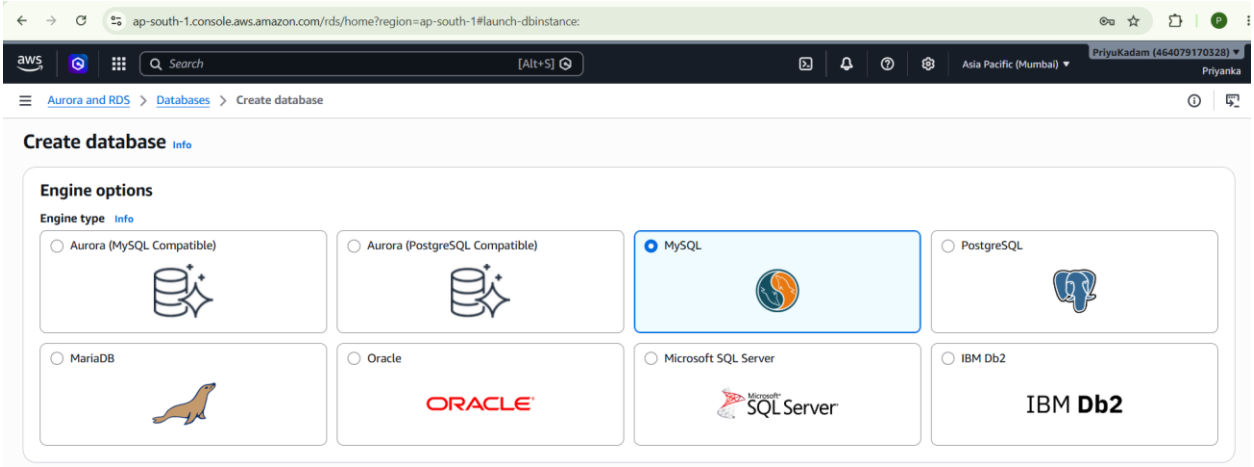

Search
[Alt+S]
Asia Pacific (Mumbai)
PriyuKadam (464079170328)
Priyanka
VPC > Your VPCs
VPC dashboard
Filter by VPC
AWS Global View
Virtual private cloud
Your VPCs
Subnets
Route tables
Internet gateways
Egress-only internet gateways
DHCP option sets
Elastic IPs
Managed prefix lists
NAT gateways
Peering connections
Route servers
Security
Network ACLs
Security groups
Your VPCs
VPCs
VPC encryption controls
Your VPCs (1/1) Info
Last updated less than a minute ago
Actions
Create VPC
Find VPCs by attribute or tag
Name
VPC ID
State
Encryption c...
Encryption control ...
Block Public...
IPv4
rds-floodify-project-vpc
vpc-08fa9df03c83ffc5b
Available
Off
10.0.0.
vpc-08fa9df03c83ffc5b / rds-floodify-project-vpc
Details
Resource map
CIDRs
Flow logs
Tags
Related resources
Integrations
Details
VPC ID
vpc-08fa9df03c83ffc5b
State
Available
Block Public Access
Off
DNS hostnames
Enabled
DNS resolution
Enabled
Tenancy
default
DHCP option set
dopt-05ea890991db4e58e
Main route table
rtb-0a678b393abfb470c
Main network ACL
Default VPC
IPv4 CIDR
IPv6 pool
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

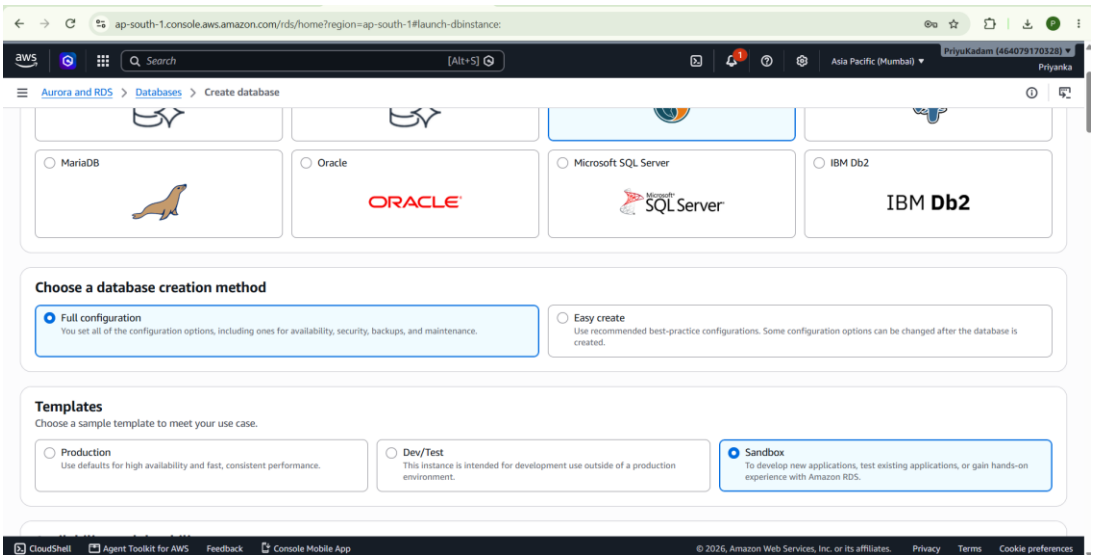
Step 2: Create the RDS Database

1. In the top search bar, search **RDS**.
2. Open **Aurora and RDS**.
3. Click **Databases** from the left menu.
4. Click **Create database –> full configuration**

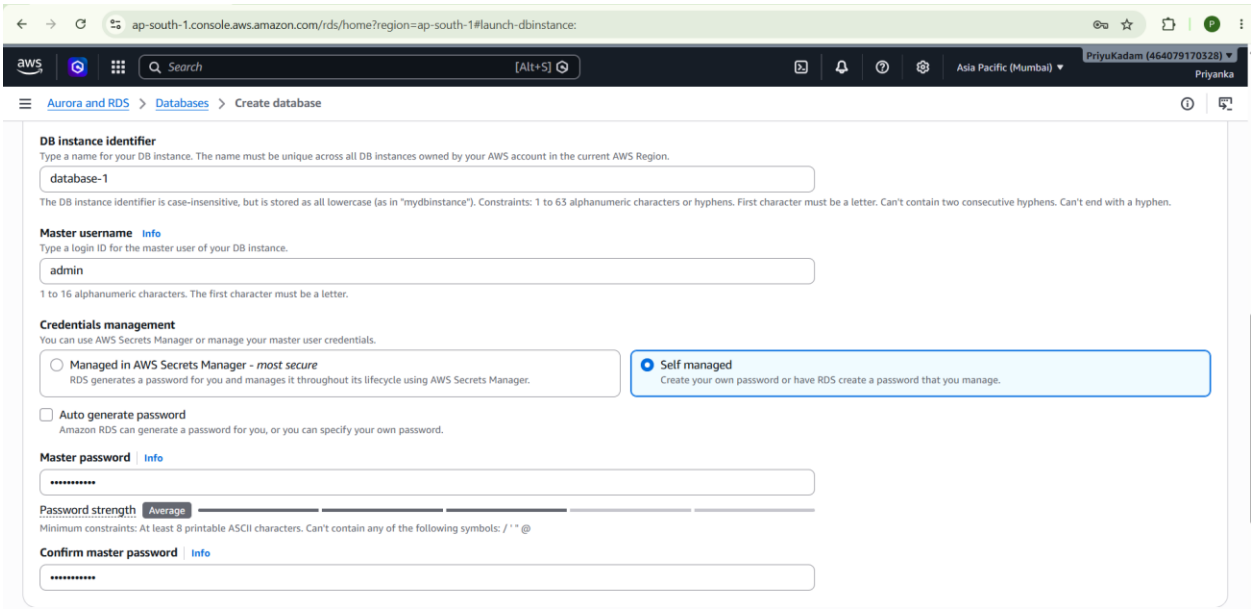


- Select the required application/database configuration

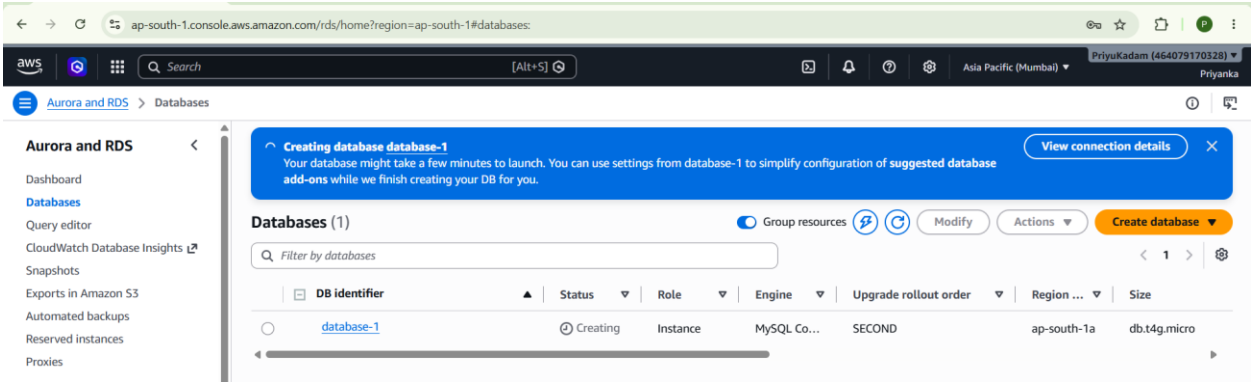




- Enter the database username and password.



- Create the database and wait until its status becomes Available.



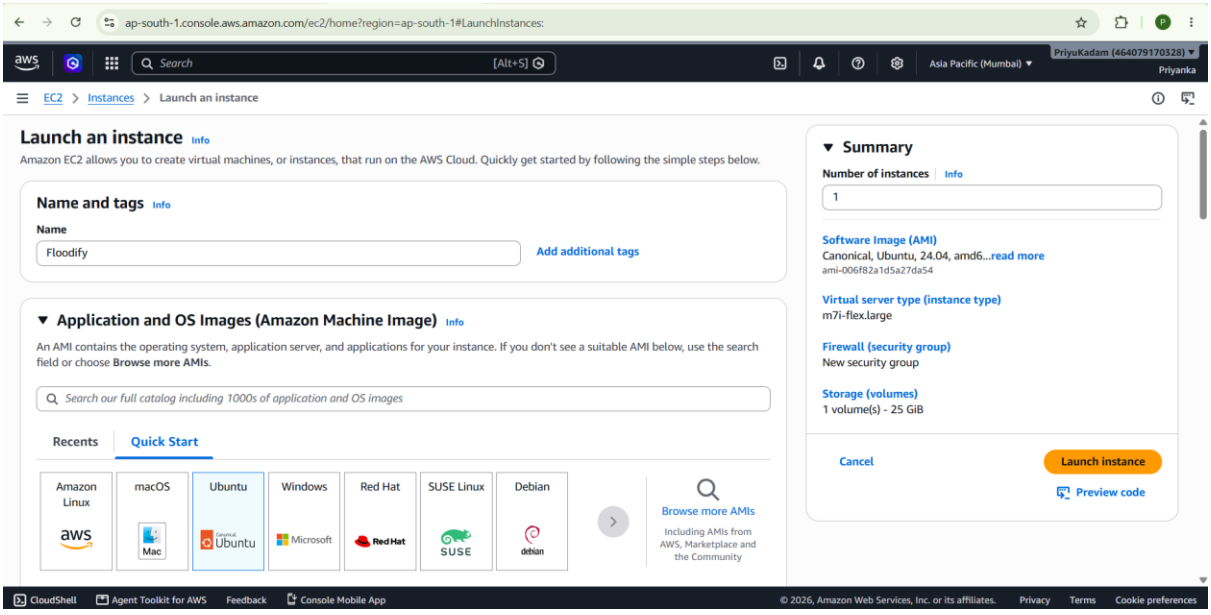
Step 3: Launch the Ubuntu EC2 Server

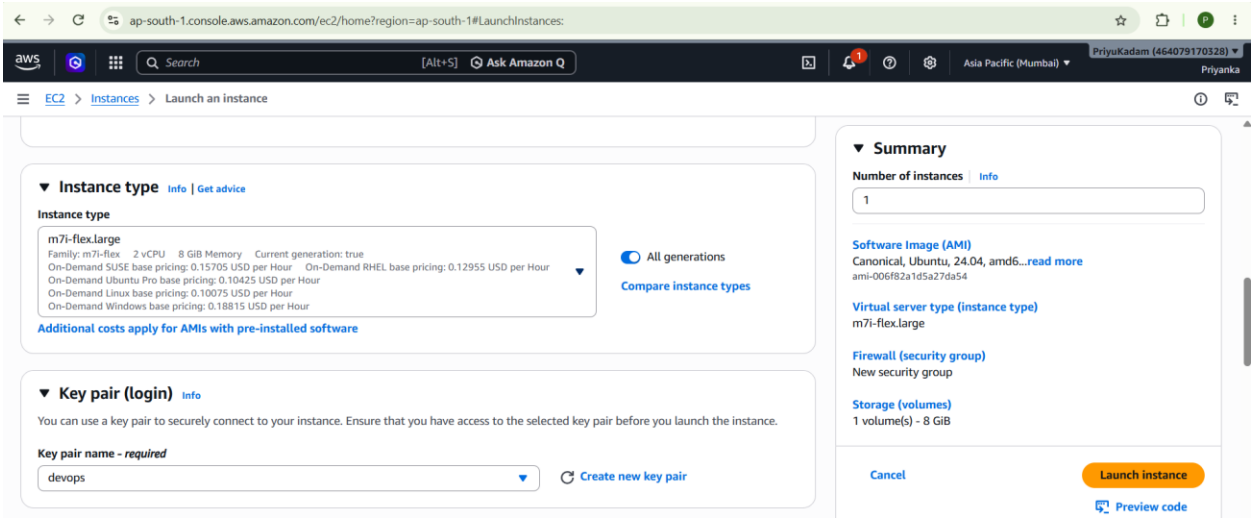
- Launch an Ubuntu EC2 instance for the application deployment.
- Configure the required storage and instance type as used in your deployment setup.
- The EC2 server will be used to run the application and connect to RDS.

Go to **AWS Console → EC2 → Instances → Launch instances**.

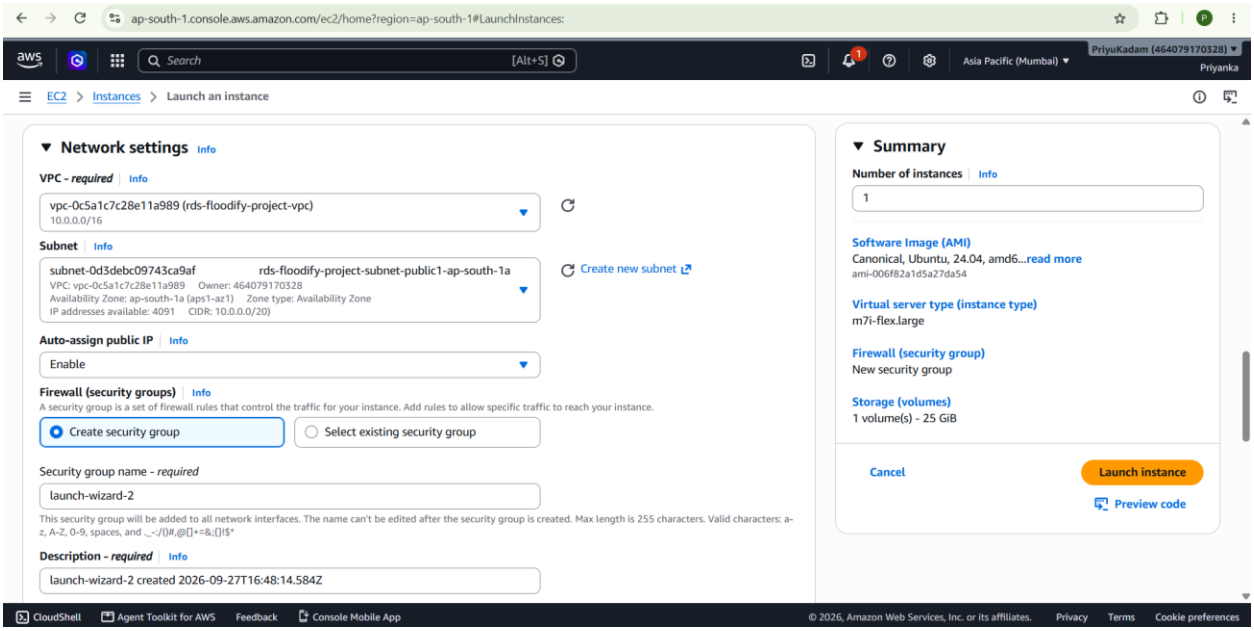
Use these settings:

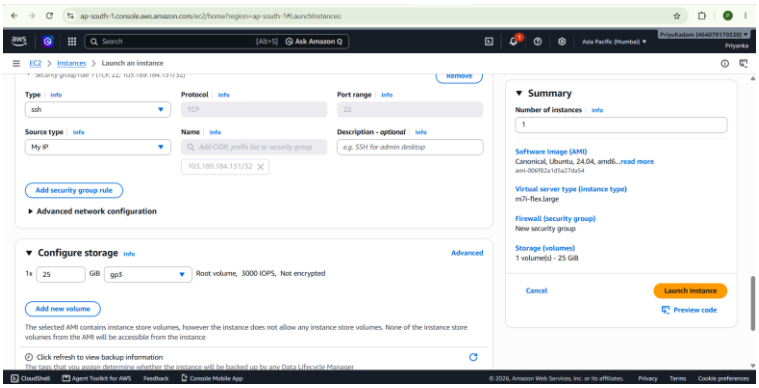
1. **Name:** Floodify
2. **Application and OS Image (AMI):**
    a. Select **Ubuntu**
    b. Choose **Ubuntu Server 24.04 LTS** (or the Ubuntu version available in your Mumbai region)
3. **Instance type:**
    a. Select **`m7i-flex.large`**
4. **Key pair:**
    a. Select an existing key pair if you already have one.
    b. Otherwise click **Create new key pair** and download the `.pem` file.
5. **Network settings:**
    a. Select your VPC.
    b. Auto assign public IP – Enable. For now, allow **SSH (port 22)** & **My IP**.
    c. Enable **HTTP (port 80)** if you will access the frontend through the browser.
6. **Configure storage:**
    a. Set **25 GiB**
7. Click **Launch instance**.





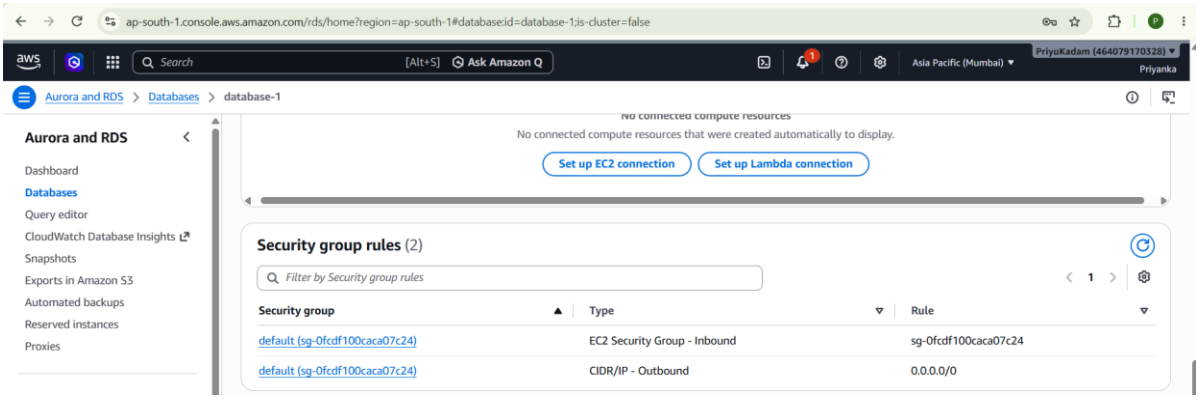
Edit Network Setting >> Select the vpc you created and public subnet and auto assign public IP >> Enable.



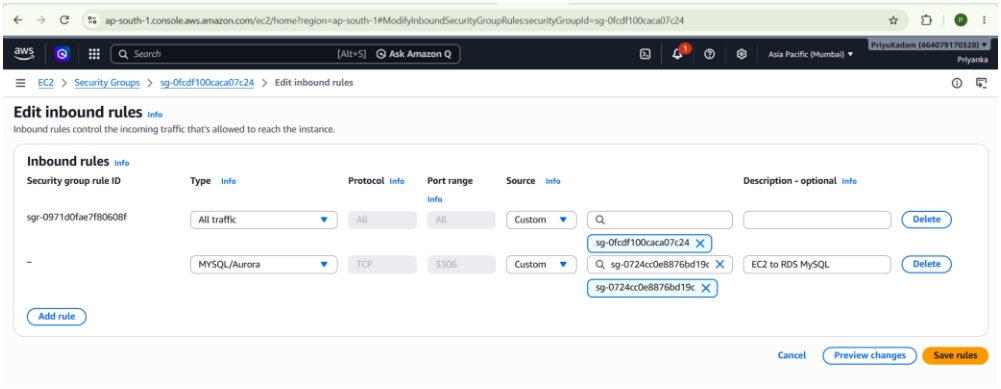


Step 4: Allow EC2 to Connect to RDS

- Go to RDS → Click on your database → Connectivity & security.
- Open the VPC security group associated with the RDS database.
- Select EC2 Security Group – Inbound



- Edit the inbound rules and allow MySQL/Aurora traffic on port 3306. >> Select the EC2 (sg-) which is attached to your EC2 security group (which u will find from EC2 Security Tab)



Step 5: Install Required Software on EC2

- Connect to the Ubuntu EC2 instance through web browser or cmd(SSH client) by copying pasting the given command in the terminal. >> cd Downloads >> SSH Command.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Priyanka K> cd Downloads
PS C:\Users\Priyanka K\Downloads> ssh -i "devops.pem" ubuntu@ec2-13-233-75-122.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-13-233-75-122.ap-south-1.compute.amazonaws.com (13.233.75.122)' can't be established.
ED25519 key fingerprint is SHA256:+gUQnyUo6bhBXO/xq81xoIAODSsSjiRdvykr2hYvT40.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-13-233-75-122.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1017-aws x86_64)
```

- Update the package list.
- Install Node.js, npm, OpenJDK 21, Maven, and MySQL client tools.
- Verify the installed versions.

Commands:

```
sudo apt update

sudo apt install nodejs npm openjdk-21-jdk maven mysql* -y

nodejs -v

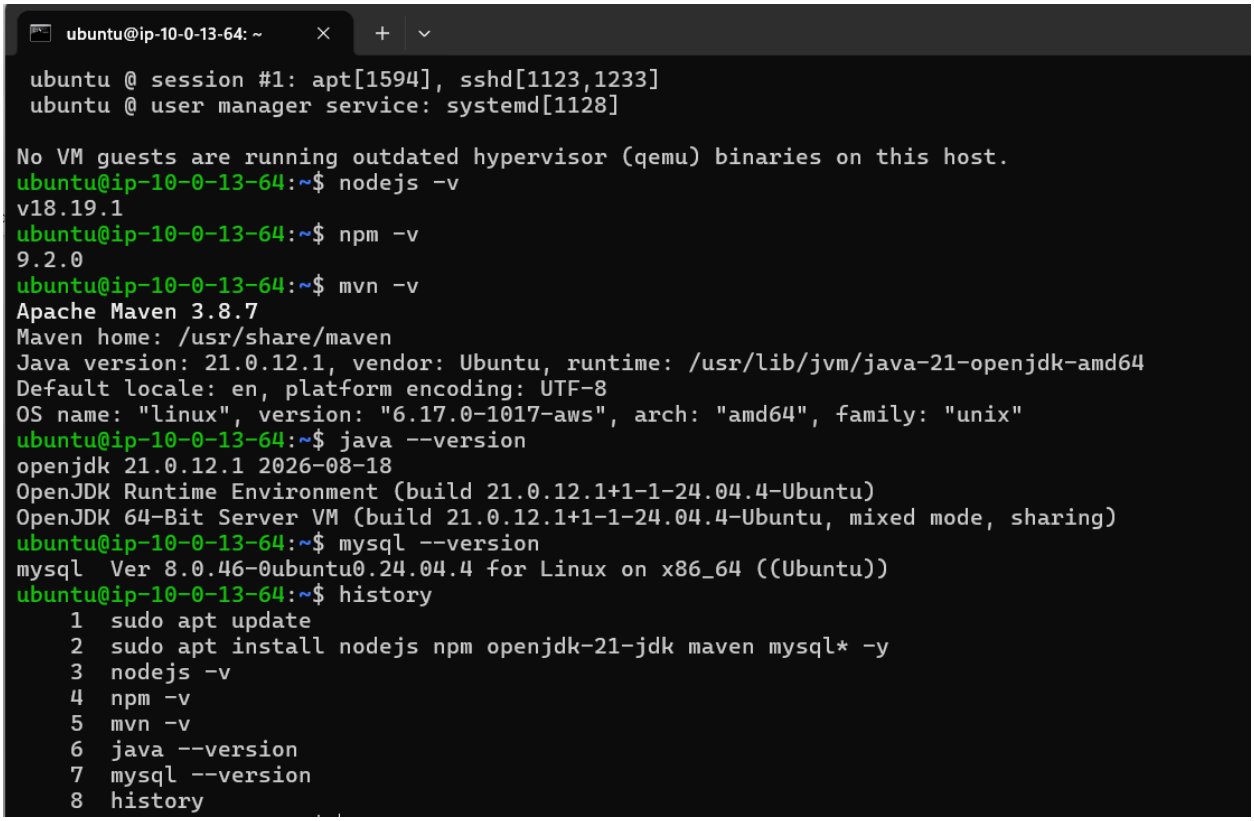
npm -v

mvn -v

java –version

mysql --version
```

Screenshot : EC2 terminal showing installation commands and version verification.



Step 6: Clone the Application Repository

- Clone the application repository.
- Move into the project directory.
- Switch to the required branch.

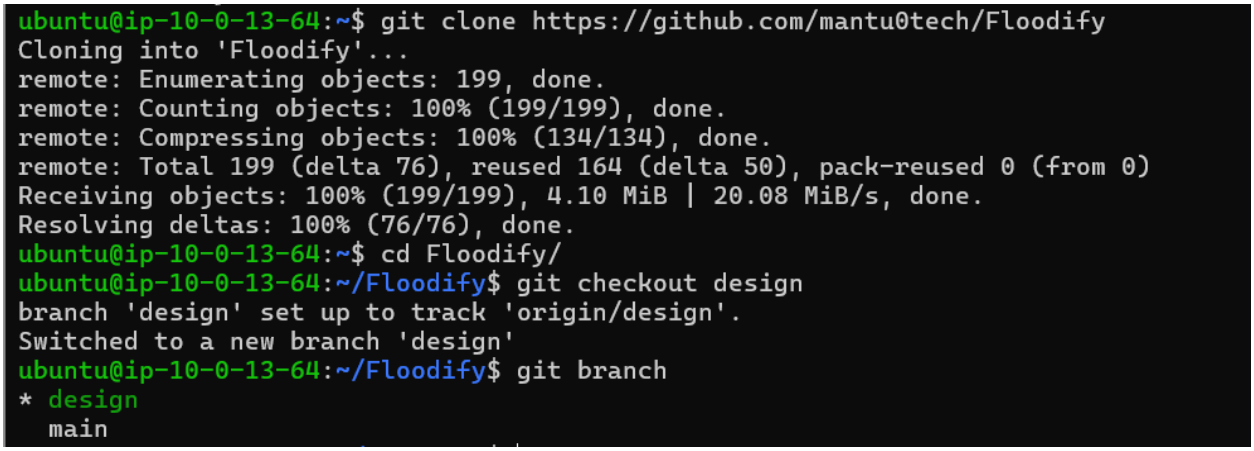
Commands:

git clone https://github.com/mantu0tech/Floodify

```
cd Floodify/
```

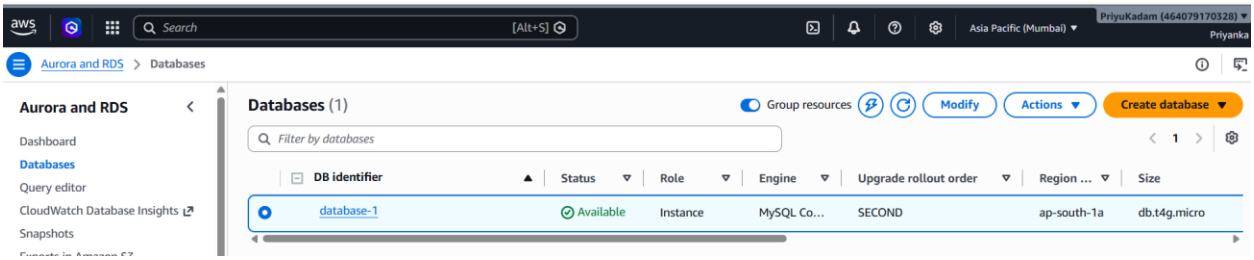
```
git checkout design
```

Screenshot: Terminal showing the git clone command, project directory, and branch checkout.



Step 7: Connect to the RDS Database

- Ensure that the RDS database status is Available.



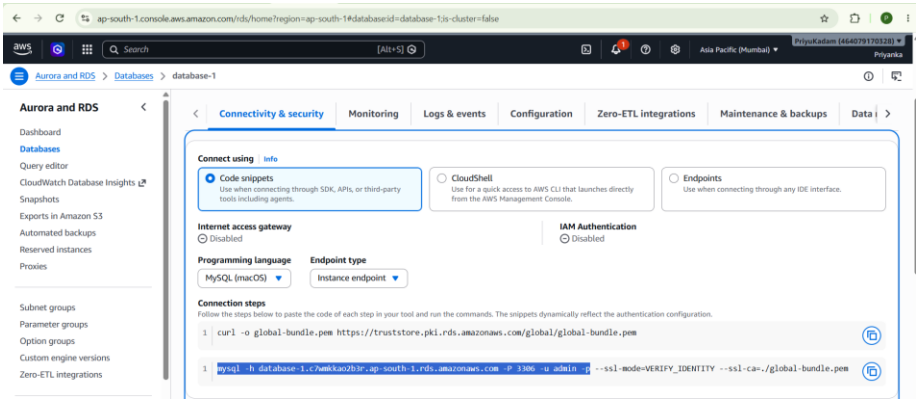
- Use the RDS endpoint/hostname, port 3306, database username, and password to connect from EC2.
- A successful connection confirms that EC2 and RDS can communicate.

  Go out of cd Floodify/ >> cd ..

Commands:

```
mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
```

Screenshot: Terminal showing successful MySQL connection to the RDS endpoint.




```
ubuntu@ip-10-0-13-64:~$ nc -vz database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com 3306
Connection to database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com (10.0.129.52) 3306 port [tcp/mysql] succeeded!
ubuntu@ip-10-0-13-64:~$ mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 85
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Please Note: Password will not be visible to you. So, prefer to type password properly.

Step 8: Import the Database SQL File

- Execute the database SQL file against the RDS MySQL database.

- This creates the required database structure and loads the required data.

Commands:

Exit;

```
ubuntu@ip-10-0-13-64:~$ mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 85
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> Exit;
Bye
```

Create the database since there is no db file created-

```
ubuntu@ip-10-0-13-64:~$ cd ~/Floodify
ubuntu@ip-10-0-13-64:~/Floodify$ mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 90
Server version: 8.4.9 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.03 sec)

mysql> CREATE DATABASE FloodifySite;
Query OK, 1 row affected (0.04 sec)

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| FloodifySite       |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.01 sec)

mysql> USE FloodifySite;
Database changed
mysql> SHOW TABLES;
Empty set (0.00 sec)

mysql> EXIT;
Bye
```

Step 9: Update the Spring Boot Database Configuration

- Replace the local database configuration with the AWS RDS endpoint/hostname.
- Configure the MySQL JDBC URL using port 3306 and the required database name.
- Save the updated application configuration.

Go to cd floodify_backend/
nano src/main/resources/application.properties and check for spring boot scripts -

```
ubuntu@ip-10-0-13-64:~/Floodify/floodify_backend$ nano src/main/resources/application.properties
ubuntu@ip-10-0-13-64:~/Floodify/floodify_backend$ cat src/main/resources/application.properties
spring.application.name=floodify_backend
spring.datasource.url=jdbc:mysql://database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com:3306/FloodifySite
spring.datasource.username=admin
spring.datasource.password=FloodifyDB123!
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

Please Note: To save nano file >> Ctrl + O >> Enter >> Ctrl + X

Step 10: Build and Run the Spring Boot Backend

- Build the backend application using Maven.
- After a successful build, run the generated JAR file.
- Verify that the backend starts successfully.

Commands:

```
mvn clean install
```




```
java -jar target/floodify_backend-0.0.1-SNAPSHOT.jar
```

```
ubuntu@ip-10-0-13-64:~/Floodify/floodify_backend$ ls -lh target/
total 52M
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 27 18:11 classes
-rw-rw-r-- 1 ubuntu ubuntu  52M Sep 27 18:12 floodify_backend-0.0.1-SNAPSHOT.jar
-rw-rw-r-- 1 ubuntu ubuntu  21K Sep 27 18:12 floodify_backend-0.0.1-SNAPSHOT.jar.original
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 27 18:11 generated-sources
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 27 18:11 generated-test-sources
drwxrwxr-x 2 ubuntu ubuntu 4.0K Sep 27 18:12 maven-archiver
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 27 18:11 maven-status
drwxrwxr-x 2 ubuntu ubuntu 4.0K Sep 27 18:12 surefire-reports
drwxrwxr-x 3 ubuntu ubuntu 4.0K Sep 27 18:11 test-classes
ubuntu@ip-10-0-13-64:~/Floodify/floodify_backend$ java -jar target/floodify_backend-0.0.1-SNAPSHOT.jar
```

Successfully backend will get started.

```
ubuntu@ip-10-0-13-64:~/Floodify/floodify_backend$ history
    1  sudo apt update
    2  sudo apt install nodejs npm openjdk-21-jdk maven mysql* -y
    3  nodejs -v
    4  npm -v
    5  mvn -v
    6  java --version
    7  mysql --version
    8  history
    9  git clone https://github.com/mantu0tech/Floodify
   10  cd Floodify/
   11*
   12  git branch
   13  cd..
   14  cd /
   15  cd ~
   16  ls
   17*
   18  ls
   19  mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
   20  nc -vz database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com 3306
   21  mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
   22*
   23  mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
   24  spring.datasource.url=jdbc:mysql://database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com:3306/FloodifySite
   25  ls
   26  find . -name "application.properties" -o -name "application.yml" -o -name "application.yaml"
   27  nano src/main/resources/application.properties
   28  nano floodify_backend/src/main/resources/application.properties
   29  cd floodify_backend
   30  mvn clean install
   31  cat src/main/resources/application.properties
   32  nano floodify_backend/src/main/resources/application.properties
   33  nano src/main/resources/application.properties
   34  cat src/main/resources/application.properties
   35  mysql -h database-1.c7wmkkao2b3r.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
   36  mvn test
   37  mvn clean install
   38  ls -lh target/
   39  java -jar target/floodify_backend-0.0.1-SNAPSHOT.jar
   40  history
```

Step 11: Run and Verify the React Frontend

- Run the React frontend after the backend and database connection are working.
- Verify that the frontend can communicate with the deployed backend.

In another terminal let's run the frontend -

```
   12  cd Floodify
   13  ls
   14  npm install
   15  cat package.json
   16  npm start
   17  history
```

Your frontend will compiled successfully after npm start.

```
Compiled successfully!

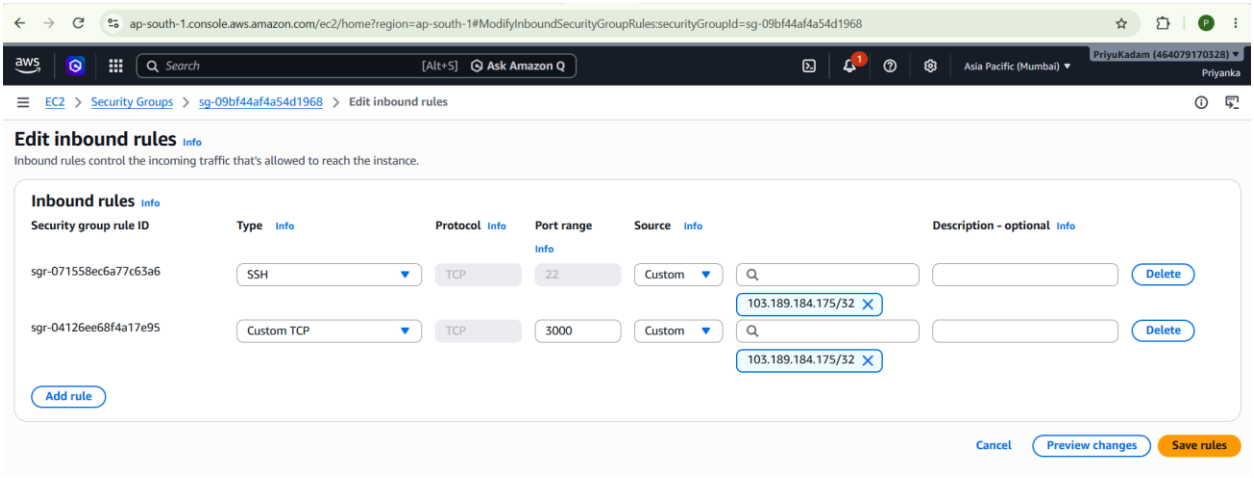
You can now view monsoon in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://10.0.12.55:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

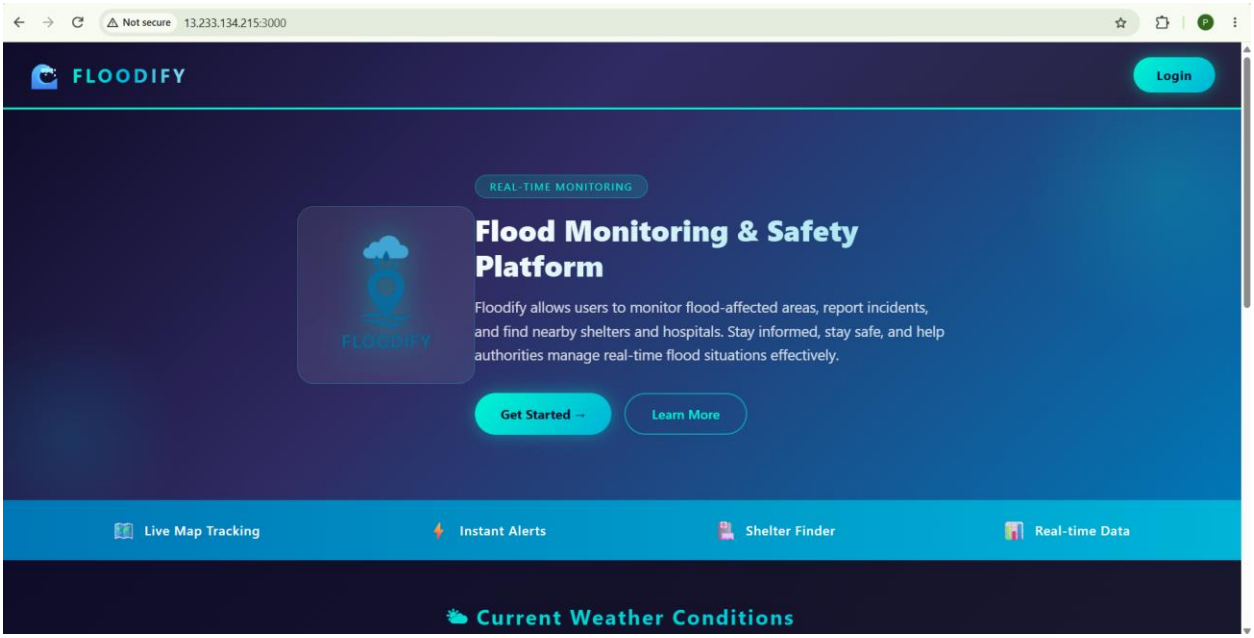
Then go to EC2 Instance – Security Tab >> Click on Security Group >> Edit Inbound rules >> add frontend port 3000



From your EC2 instance access your public IP4 address -

EC2 instance >> Details Tab >> Access your public ipv4 address

For example - http://13.233.134.215:3000/



Step 12: Cleanup

- After completing the deployment practice, delete or stop AWS resources that are no longer required.

Final Architecture

React Frontend → Spring Boot Backend on EC2 → MySQL Database on AWS RDS

The application runs on the EC2 server, while the MySQL database is hosted separately on AWS RDS.